SMART STUDY RECALL / ENGINEERING AND RESEARCH GUIDE

System Architecture and Operating Guide

Services, APIs, storage, security, setup and learner data flow

Prepared from the repository implementation and experiment reports as inspected on 27 October 2026.

Faculty presentation edition | Implementation facts are separated from research hypotheses and unvalidated claims.

How to use this document

This guide explains the running implementation, what data enters each component, what algorithms do with it, what is saved, how to operate the project, and what the current experiments do and do not establish. Formulae use plain ASCII notation. Source paths at the end let a reviewer trace each description back to the repository.

Important interpretation rule: a predicted next-session correct fraction is a score proxy. It is not a measured probability that a learner will remember a course concept on a future date.

Contents

01. 1. Runtime architecture

02. 2. Routes and responsibility boundaries

03. 3. Main persisted entities

04. 4. Environment and startup

05. 5. What happens in one study session

06. 6. Database and file extraction

07. 7. Operational checklist and troubleshooting

1. Runtime architecture

The application has a static client, a main authenticated app API and a separate ML API. Ports in the local guide are 5500, 8002 and 8000, respectively. PostgreSQL and Supabase Auth are remote services. The frontend uses browser JavaScript with no build step.

Live Server :5500 HTML + JS	FastAPI app :8002 Auth + business logic	ML FastAPI :8000 Model inference	Supabase Auth + PostgreSQL

A request to a protected app route carries Authorization: Bearer <access_token>. The backend asks Supabase Auth to identify that user, maps the Supabase user ID to a Profile, then enforces profile ownership before reading or writing student records. The ML endpoint is designed for local use and does not replace app authentication.

2. Routes and responsibility boundaries

Route group	Purpose	Important behavior
/students	Resolve current profile and account operations	Bearer token required
/topics	Create/list/update/delete learner topics	Owned by authenticated student
/learning-events	Save and list study outcomes	Correct answers cannot exceed questions seen
/revision-schedules	Create, complete and reschedule dates	Timezone-bearing timestamps
/recommendations	Model/rule priorities and automatic schedule policy	Model score prediction is separate from date policy
/dashboard	Aggregate metrics and pending schedule counts	Student-scoped
/materials	Upload/list/delete source files; notes/quiz/chat	Extracted text is stored as chunks
/rag/{id}	Architecture, chat, quiz, notes, history and retrieval comparison	Material access checked; interaction data saved after migration 004
ML /predict/next-session/scenarios	Score predictions at candidate intervals	Bounded gaps, up to 30 days
ML /predict/knowledge-tracing	MLP/LSTM/Transformer candidate inference	Up to 500 time-ordered interactions

A good design boundary is that the main app API owns authentication, authorization and persistence. The model service accepts only the necessary features and returns a score. The browser does not access PostgreSQL directly.

3. Main persisted entities

Profiles map Supabase identities to app-specific display information. Topics belong to profiles. LearningEvent rows are append-only study outcome records: questions_seen, questions_correct, event_time and optional topic_id. RevisionSchedule rows store scheduled_at, completed and rescheduled_count. Material records link uploaders to chunk rows. Quiz response rows store concept and correctness, which feeds adaptive RAG heuristic state.

Browser-local focus minutes and flashcards are different: they live in localStorage on a specific browser profile/origin, so clearing site data or using another browser loses access. Focus minutes are not included in model training or prediction features.

RAG chat interaction history requires migrations/004_rag_chat_history.sql. Run migrations in the order described in README. A model artifact file being present and a corresponding database schema existing are separate deployment concerns.

---

4. Environment and startup

```
Repository-root .env
DATABASE_URL=postgresql+psycopg2://...
SUPABASE_URL=https://<project-ref>.supabase.co
SUPABASE_ANON_KEY=<public-anon-key>
ML_SERVICE_URL=http://127.0.0.1:8000
FRONTEND_ORIGINS=http://127.0.0.1:5500,http://localhost:5500
AI_API_KEY=<optional provider key>
AI_API_BASE_URL=<provider-compatible endpoint>
AI_MODEL=<text generation model>

frontend/js/config.js
SUPABASE_URL=<same project URL>
SUPABASE_ANON_KEY=<same public anon key>
API_URL=http://127.0.0.1:8002
```

- Keep the anon key public by design but do not place the service_role key, database URL or AI provider secret in frontend code. Restart the backend after changing .env because settings are read at process startup.

Start from the repository root. The ML service command is run after changing into ml/src because its imports are module-local. If a port is already occupied, stop the existing process rather than silently connecting to an older server.

---

5. What happens in one study session

Learner submits quiz/session	POST /learning-events	Persist event in PostgreSQL	Planner/forecast reads ordered events	ML service returns score scenarios

The frontend computes questions seen and correct from the quiz result and sends them with the topic ID. The API validates ownership and count consistency. The recommendation route groups events by topic and orders them by event time. The adapter transforms them into the ML model contract, inserts the candidate gap for the most recent event and sends the request to localhost:8000.

The model returns predicted next-session correct fraction at each candidate gap. A separate policy picks an interval that satisfies the declared 80% target where possible. When forecast output is unavailable, the schedule endpoint has a conservative rule fallback. An automatic planner operation schedules only topics with an available model result and no unresolved cycle; the learner still sees the schedule in calendar and agenda.

---

6. Database and file extraction

Uploaded formats are PDF, DOCX, TXT and Markdown. Text PDFs are extracted page by page; DOCX paragraphs and tables are included. Encrypted PDFs are rejected. Scanned image-only PDFs need OCR before upload. Files are capped at 12 MB and extracted content at 1.2 million characters.

The backend creates overlapping word-window chunks: 210 words per chunk with a 35-word overlap. It stores the material metadata and each chunk's content, index and top keyword list in PostgreSQL. The implementation does not write dense embedding vectors to pgvector or another vector database.

RAG chat history saves the prompt, answer, mode, retrieval method, citations, diagnostics and an optional user support rating. It is scoped by authenticated student and material. Apply migration 004 before the backend uses this table.

---

7. Operational checklist and troubleshooting

- The dashboard error 'Could not verify Supabase authentication' means the main API could not verify the browser bearer token with Supabase Auth. Check that backend and frontend project URLs match, the project is active, and the Python process has outbound access to Supabase. The project URL is not the database pooler URL.
- A 404 on /rag/{material_id}/history with a successful material architecture call usually means the running app backend is older than the current frontend. Restart the main API after code changes. A 500 or missing-table message points to migration 004 instead.
- A 502 from note/chat generation can indicate provider timeout, quota or network failure. The app has an extractive path when no model is configured, but an explicitly configured provider failure can surface as an error. Keep keys in .env and confirm AI_MODEL is an actual available model name.
- A missing main recall forecast can mean the ML service is stopped, the champion artifact is absent, the data is outside the supported interval or no study event exists. Check ML /health and inspect the reason returned by the model adapter.

A presentation demo should use a topic with at least one valid session event, an available ML service, a future review date, and a short text-based course document. Verify each dependent service separately before the presentation.

Repository traceability and source files

The report describes the local repository snapshot, not an independently deployed cloud instance. A running deployment can differ if its backend, database migrations, model artifacts or environment variables have not been updated.

Primary source files reviewed:

- README.md
- docs/API.md
- docs/SETUP.md
- app/main.py
- app/auth.py
- app/models.py
- app/routers/
- frontend/js/api.js
- migrations/